AI Compiler Performance Engineering

Auto-Loops Books — ch17

Contents

1 OpenAI Triton: Pythonic GPU Kernel Compiler **1**

1.1 Triton HW matrix 1

1.2 Triton arch passes 4

1.3 Triton codegen 8

1.4 Triton tuning 11

1.5 Triton case study 13

1.6 Key Takeaways 16

1.7 Conclusion 17

Bibliography **18**

Index **20**

Chapter 1

OpenAI Triton: Pythonic GPU Kernel Compiler

Triton made GPU kernel authoring look like Python and still hit cuBLAS/cuDNN-class throughput, which is exactly why it is dangerous in a decode pipeline if you stop at "the kernel is fast" (Tillet *et al.*, 2019). The language is built around one abstraction—the *tile*, a statically shaped multi-dimensional sub-array—so a programmer writes a per-block program over tiles and the compiler handles coalescing, shared-memory staging, and instruction scheduling (Tillet *et al.*, 2019). That productivity is real: FlashAttention-style fused attention is practical to write and maintain in Triton in a way it never was in raw CUDA (Dao *et al.*, 2022). The decode trap is the same one Chapter 16 raised for TVM. A beautiful Triton matmul does nothing for batch-1 latency if the decoder layer still dispatches a dozen separate kernels per token. Dense Llama-3.2-1B already averages 847.5 GPU kernel launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). Triton helps only when it is used to *fuse* the decode loop into fewer, residency-aware kernels, and when those kernels are captured to amortize host launch cost (Chen, 2026; NVIDIA, 2024). This chapter treats Triton as a block-level kernel compiler graded on bytes/token and launches/token, not as a Python tutorial (Davies *et al.*, 2025).

1.1 Triton HW matrix

Triton's support **matrix** is deliberately narrower than a general tensor compiler's, and pretending otherwise is the first **hardware** mistake. Triton is GPU-first: the

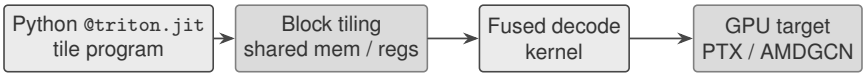


Figure 1.1: Triton compiles a per-block Python tile program to a fused GPU kernel; the block model assumes a GPU memory hierarchy.

production back-ends emit NVIDIA PTX and AMD AMDGCN, with CPU and Intel GPU back-ends emerging (OpenAI / triton-lang contributors, 2024). Its programming model assumes a SIMT machine with warps, shared memory, and a coalesced global-memory **target**—so it maps cleanly onto GPUs and not onto static-dataflow fabrics (NVIDIA, 2022; Tillet *et al.*, 2019).

The honest multi-hardware reading is that Triton is the right abstraction on one class and the wrong one on another. On a **gpu** it lets you express residency—which tiles live in shared memory, how loads are pipelined—at exactly the granularity decode needs (Tillet *et al.*, 2019; Chen, 2026). On a **cpu** the same block program can be lowered but loses its raison d'être, since cache blocking and SIMD want a different schedule (Goodfellow *et al.*, 2016). On an **npu** such as XDNA/AIE the block-over-warps model does not apply at all: those fabrics demand static tile-to-DMA assignment decided at compile time, not a SIMT block grid (AMD, 2024b,a).

Triton's design point also explains where it fits between raw CUDA and a whole-graph compiler. It abstracts away intra-block concerns—thread indexing, explicit shared-memory management, and memory coalescing are the compiler's job, not the author's—while leaving inter-block tiling and the fusion boundary to the programmer (Tillet *et al.*, 2019). That is more control than a graph compiler that decides fusion for you and more productivity than CUDA where you manage every load by hand. For decode the consequence is specific: Triton gives you exactly the knob that matters—where the fusion boundary sits—but it does not decide that boundary for you, so a careless kernel can still be fast in isolation and wasteful in a layer (Vellaisamy *et al.*, 2026; Chen, 2026).

Multi-hardware delta. A Hopper GPU lets a Triton kernel hide latency with asynchronous copy and pipelined loads; an XDNA NPU cannot run the block model and needs a different compiler path entirely (SemiAnalysis, 2024; AMD, 2024a). This is why a serving stack treats Triton as one back-end among several rather than a universal target, and why YiRage carries Triton as a GPU code path while routing NPU work elsewhere (Chen and YiRage Contributors, 2026).

1.1.1 Where Triton sits in the decode stack

Triton occupies a specific layer: below the graph compiler that decides fusion, and above the CUDA code the device finally runs (Tillet *et al.*, 2019; Chen, 2020). A graph compiler such as XLA or IREE can lower a fused region into Triton source, or a team can write the Triton kernel directly and call it from a serving runtime (OpenXLA Project, 2024; OpenAI / triton-lang contributors, 2024). The direct route gives the team complete control over the fusion boundary, while the compiler route gives them the boundary that the graph passes found legal (Davies *et al.*, 2025; Chen, 2026). Decode teams need to know which route their stack uses because the debugging surface differs: direct kernels are debugged in isolation and integrated by hand, while compiler-emitted Triton is debugged by inspecting the graph passes that produced it (Chen, 2020; Davies *et al.*, 2025). Both routes still feed the same counters this book uses, and both must be measured at the batch-1 decode operating point rather than on the author's favorite benchmark shape (Vellaisamy *et al.*, 2026; Chen, 2026).

1.1.2 Triton, torch.compile, and Inductor

Most PyTorch users meet Triton through torch.compile, whose Inductor backend emits Triton kernels for fused regions instead of requiring hand-written CUDA (Davies *et al.*, 2025; Chen, 2020). That path inherits Triton's strengths—block-level codegen, automatic coalescing, software pipelining—and its limits: Inductor decides the fusion boundaries from the graph it receives, so an eager model with attention broken into framework ops produces different Triton kernels than a model whose attention is already a fused custom region (Dao *et al.*, 2022; Davies *et al.*, 2025). For decode, torch.compile can collapse elementwise and projection chains, but it does not automatically invent online softmax or a persistent decoder kernel if the source graph does not contain that structure (Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). The engineering takeaway is that Inductor is a productive starting point whose output must be audited with the same TTGIR and PTX diffs as hand-written Triton, and whose fusion boundaries are only as good as the export that feeds them (Chen, 2020, 2026).

1.1.3 GPU generation differences

The Triton source may be hardware-agnostic, but the schedule that makes it fast is not (OpenAI / triton-lang contributors, 2024; NVIDIA, 2022). Hopper and Blackwell GPUs differ in shared-memory capacity per SM, tensor-core MMA

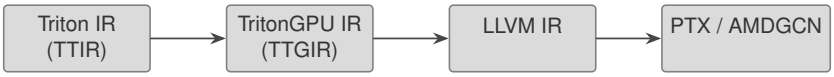


Figure 1.2: Triton's MLIR pass pipeline: the Triton dialect (TTIR) is lowered to the TritonGPU dialect (TTGIR), then to LLVM and a GPU assembly target.

shapes, async-copy depth, and warp-specialization support, so a kernel with one `num_stages` and layout configuration can be optimal on one generation and spill or stall on the next (SemiAnalysis, 2024; NVIDIA, 2022). The same applies across AMD GPUs with different vector widths and cache hierarchies (OpenAI / triton-lang contributors, 2024). A fleet with mixed GPU generations should therefore compile and autotune per SKU, store the winner beside the SKU identifier, and never copy a cache entry from another generation (Chen, 2026; Davies *et al.*, 2025). The build matrix for Triton is small in vendor count but large in schedule diversity, and the schedule diversity is where decode bytes/token is won (Vellaisamy *et al.*, 2026; Chen, 2026).

1.1.4 Portability expectations beyond NVIDIA

Triton's AMD backend demonstrates that the block abstraction is portable in syntax but not in performance outcomes (OpenAI / triton-lang contributors, 2024). The same kernel emits AMDGCN with different vector widths, cache behavior, and instruction selection, so a config tuned on NVIDIA can be far from optimal on AMD even when it compiles cleanly (OpenAI / triton-lang contributors, 2024; Goodfellow *et al.*, 2016). Emerging CPU and Intel GPU back-ends widen the matrix further but inherit the fundamental mismatch: Triton's tile-over-warps model assumes a SIMT memory hierarchy, which is precisely what CPUs and static NPUs do not provide (AMD, 2024b; Goodfellow *et al.*, 2016). A multi-vendor fleet should therefore treat each Triton backend as a separate qualification cell with its own autotune cache and benchmark artifacts (Davies *et al.*, 2025; Chen, 2026). Portability in Triton means the source expresses the algorithm once; the schedule, the cache, and the qualification still belong to each backend (Tillet *et al.*, 2019; Vellaisamy *et al.*, 2026).

1.2 Triton arch passes

Modern Triton's backend was rewritten on MLIR, so the **pass** pipeline is an explicit **dialect** tower: the Triton **dialect** (TTIR) captures the tile program;

Table 1.1: Triton's MLIR lowering stages and what each level controls; the TTGIR stage is where decode residency is decided (OpenAI / triton-lang contributors, 2024).

Stage	Dialect / form	What it controls
TTIR	Triton dialect	Tile program, target-agnostic
TTGIR	TritonGPU dialect	Layout, coalescing, pipelining
LLVM IR	LLVM	Scalar/vector lowering, registers
PTX / AMDGCN	GPU assembly	Final emit (`ptxas` / ROCm)

the TritonGPU dialect (TTGIR) attaches GPU-specific layout and scheduling; both then **lower** to LLVM IR and a GPU assembly target (OpenAI / triton-lang contributors, 2024). Unlike TVM's fixed two-level IR, Triton commits to this progressive-lowering dialect stack and concentrates its **optim**ization in the TTGIR stage (Chen, 2020; Tillet *et al.*, 2019).

The TTGIR passes are where decode residency is won or lost. Layout assignment decides whether a tile lives in registers, shared memory, or an MMA-friendly layout for tensor cores; coalescing passes decide whether global loads are contiguous; the software-pipelining pass overlaps loads with compute by issuing them `num_stages` iterations ahead (OpenAI / triton-lang contributors, 2024; SemiAnalysis, 2024). Get the layout pass wrong and the kernel silently spills a tile to global memory between blocks—which reads downstream as an unexplained bytes/token regression rather than a compile error (Chen, 2026; Vellaisamy *et al.*, 2026).

Triton's layout system is the part most CUDA programmers underestimate. A blocked layout distributes a tile across threads for coalesced loads; an MMA layout matches the register fragment shape a tensor-core instruction expects; a shared layout stages data for cross-thread reuse (OpenAI / triton-lang contributors, 2024). The compiler inserts conversions between layouts automatically, and an unnecessary conversion is a hidden shared-memory round trip—cheap in a throughput kernel, measurable in a batch-1 decode kernel that runs thousands of times per token window (Vellaisamy *et al.*, 2026; SemiAnalysis, 2024). This is the level at which fusion either keeps the decoder layer on-chip or quietly does not.

What to inspect before merge. Triton can dump every stage (`TRITON_KERNEL_DUMP`, `MLIR_ENABLE_DUMP`); diff the TTGIR and the emitted PTX for new shared-memory allocations or un-pipelined loads, and count launches/token at the call site (OpenAI / triton-lang contributors, 2024; NVIDIA, 2024). A pass reorder that breaks pipelining will not change correctness, only p99 (Davies *et al.*, 2025).

1.2.1 Layout assignment as the residency decision

The TTGIR layout assignment pass is the single most consequential step between a Triton program and the emitted GPU code because it decides where every tile physically lives (OpenAI / triton-lang contributors, 2024; SemiAnalysis, 2024). Register layouts suit values consumed immediately by arithmetic; shared layouts suit values reused across warps or staged by async copies; MMA layouts match the exact fragment shape a tensor-core instruction consumes (OpenAI / triton-lang contributors, 2024; NVIDIA, 2022). When the compiler cannot assign one layout to a tile across its whole lifetime it inserts a layout conversion, which for large tiles means a round trip through shared memory (OpenAI / triton-lang contributors, 2024; Chen, 2026). A decode kernel that runs per token thousands of times cannot hide those conversions behind other work the way a prefill kernel can, so layout conversions are a first-order p99 cost (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Reviewers should diff the TTGIR layout annotations and treat newly inserted conversions as regressions until proven otherwise (Chen, 2020, 2026).

1.2.2 Coalescing and memory access patterns

Triton promises the author automatic coalescing, but the promise is conditional on the block sizes and pointer patterns the program expresses (Tillet *et al.*, 2019; OpenAI / triton-lang contributors, 2024). A tile whose logical layout does not match the physical addresses being loaded will produce scattered transactions even when the compiler applies its best vectorization (SemiAnalysis, 2024; Chen, 2026). At batch-1 decode, KV-cache reads are the canonical example: block tables and paged layouts make the address stream non-contiguous, and the Triton kernel must express the gather or use a page-consistent tile shape so the coalescing passes have something contiguous to work with (Kwon *et al.*, 2023; Davies *et al.*, 2025). The emitted PTX should be inspected for vector-width eight-byte or sixteen-byte loads where the source expected them; scalarized loads are the signature of a layout mismatch that no autotuner config will fix (OpenAI / triton-lang contributors, 2024; Chen, 2026).

1.2.3 Software pipelining and the decode shape

The `num_stages` pass overlaps global loads with computation by prefetching future tiles, which is exactly what a memory-bound attention kernel needs at long context (OpenAI / triton-lang contributors, 2024; SemiAnalysis, 2024). Each added stage costs shared memory, so the maximum legal depth is set by the tile size and the

GPU's shared-memory capacity, not by the programmer's preference (NVIDIA, 2022; Chen, 2026). At batch-1 decode, a deep pipeline is usually wrong: the KV loop is long enough to hide latency, but the shared-memory cost of four stages can cut occupancy on the thin attention GEMMs that follow (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). The right depth therefore depends on context length, tile size, and SKU, which makes `num_stages` an autotune axis rather than a constant in the source (OpenAI / triton-lang contributors, 2024; NVIDIA, 2022). When profiling shows shared-memory-related occupancy loss, halve the stages before shrinking the block, because fewer stages preserve the larger transaction width that decode needs (Chen, 2026; SemiAnalysis, 2024).

1.2.4 TTGIR debugging workflow

The practical way to find a layout or pipelining regression is to make the IR visible at every stage and diff it across changes (OpenAI / triton-lang contributors, 2024; Chen, 2020). Enable the Triton and MLIR dump flags, capture the TTIR and TTGIR for the decode kernel, and archive them with the source revision (OpenAI / triton-lang contributors, 2024). When a Triton upgrade changes behavior, diff the TTGIR before and after: new layout conversions, changed pipelining order, or different shared-memory allocations identify the regression without guesswork (Chen, 2026; Davies *et al.*, 2025). The same dump workflow catches authoring mistakes such as masks that defeat vectorization or reductions that force shared-memory round trips (OpenAI / triton-lang contributors, 2024; Chen, 2026). A team that diffed the IR first will spend minutes on regressions that cost other teams days of profiler archaeology (Vellaisamy *et al.*, 2026; Chen, 2020).

1.2.5 Masks, padding, and vectorization

Triton programs handle variable lengths with masks and padding, and both interact with the compiler's vectorization and layout decisions (OpenAI / triton-lang contributors, 2024; Kwon *et al.*, 2023). A mask that is uniform across the tile lets the compiler specialize and keep vector loads; a per-element mask on a paged KV walk can force scalarization or predication that defeats coalescing (OpenAI / triton-lang contributors, 2024; Chen, 2026). Decode kernels should prefer structure that makes masks cheap: bucket context lengths, pad at tile boundaries, and arrange the page layout so masked lanes sit at tile edges (Kwon *et al.*, 2023; Davies *et al.*, 2025). The cost of padding is capacity waste, and the cost of masks is throughput loss, so the block size choice is a balance between the two at the served context distribution (Chen, 2026; Vellaisamy *et al.*, 2026). When profiling

shows low vector width on a masked kernel, restructure the data layout before tuning block shapes, because no config fixes scalarized loads (OpenAI / triton-lang contributors, 2024; Chen, 2026).

1.3 Triton codegen

Codegen in Triton is per-GPU and explicit. The NVIDIA **backend** lowers LLVM IR to PTX and runs `ptxas` to **emit** a CUBIN; the AMD backend emits AMDGCN for ROCm (OpenAI / triton-lang contributors, 2024). The **platform** you target therefore changes the final assembly, but the controlling knobs are the same two parameters that govern the emitted schedule: `num_warps` (warps per CTA, hence threads = `num_warps`×32) and `num_stages` (software-pipelining depth) (OpenAI / triton-lang contributors, 2024).

The trap is grading the emitted kernel by its standalone throughput. At batch-1 decode the dominant cost is host orchestration and surviving HBM round trips, not the quality of a single emitted MMA (Vellaisamy *et al.*, 2026; Chen, 2026). Two codegen decisions actually move ms/token. First, fuse: emit one kernel for the decoder layer (e.g. a Triton FlashAttention plus the projection epilogue) so activations never leave the chip between operators (Dao *et al.*, 2022). Second, capture: wrap the fused launch sequence in a CUDA Graph so the host stops paying per-kernel dispatch every token—the Memory-Floor study recovered roughly 1.26× on an H100 Qwen-2.5-7B decode cell from capture alone, with no change to the emitted math (Chen, 2026; NVIDIA, 2024).

There is a second-order codegen cost that decode exposes: register pressure. The LLVM stage allocates registers per thread, and a kernel that fuses an entire decoder layer can exceed the per-thread budget, forcing spills to local memory or capping occupancy through `maxnreg` (OpenAI / triton-lang contributors, 2024; NVIDIA, 2022). On a throughput kernel low occupancy is often fine because each block does enough work; on a batch-1 decode kernel the launch is already thin, so a register spill is pure overhead with nothing to hide behind (Vellaisamy *et al.*, 2026; Chen, 2026). The practical consequence is that "fuse everything" is not free—past a point, a slightly less fused kernel with healthy occupancy beats a maximally fused one that spills.

Multi-hardware delta. Raising `num_stages` prefetches more aggressively and hides latency on a Hopper GPU with deep async-copy queues, but it costs shared memory and can reduce occupancy on a smaller SKU (SemiAnalysis, 2024; NVIDIA, 2022). The same Triton source therefore wants a different `num_stages` per GPU

generation, and none of it transfers to a static NPU fabric (AMD, 2024b).

1.3.1 Compile caching and ptxas

Triton compiles each kernel configuration through LLVM and `ptxas` at first use and caches the result keyed by source hash, target, and config (OpenAI / triton-lang contributors, 2024). The cache is a deployment surface: a cold cache turns a serving pod's first requests into compile stalls, while a warm cache hides compile time entirely (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Production stacks should precompile the exact kernel set for the shipped model and SKU at build time and ship the cache or compiled artifact with the release (Chen, 2020, 2026). The cache key must include the Triton version, LLVM version, and target string, because any one of them can change the emitted PTX and invalidate a performance assumption (OpenAI / triton-lang contributors, 2024; Davies *et al.*, 2025). Teams that let the cache build itself on first request are choosing warmup latency as an implicit product cost (Kwon *et al.*, 2023; Chen, 2026).

1.3.2 Register budget as a fusion constraint

Register pressure is the wall that limits how much of a decoder layer one Triton kernel can fuse (NVIDIA, 2022; OpenAI / triton-lang contributors, 2024). The LLVM stage allocates a physical register file per thread, and a fused kernel holding attention state, projection tiles, and MLP intermediates simultaneously will either spill to local memory or force the compiler to serialize register lifetimes (OpenAI / triton-lang contributors, 2024; Chen, 2026). Spills to local memory look like DRAM traffic in profiles and are frequently misdiagnosed as bandwidth regressions (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). The tuning response is to inspect the PTX register count and the ptxas spill report before changing anything else: if the kernel spills, reduce fusion width or tile size before tuning block shapes (OpenAI / triton-lang contributors, 2024; NVIDIA, 2022). A kernel that fits registers at batch-1 attention can still spill when the same source is generalized to a longer context tile, which is why register review belongs in the per-bucket qualification record (Chen, 2026; Chen and YiRage Contributors, 2026).

1.3.3 Inspecting emitted code

Triton exposes the full IR stack for inspection, and decode teams should use it on every kernel that will run per token (OpenAI / triton-lang contributors, 2024). The workflow is to dump TTIR, TTGIR, LLVM IR, and PTX; verify that

the fused region the author intended actually survived as one kernel; check the TTGIR for layout conversions and shared-memory allocations; and confirm the PTX contains the expected vector loads and MMA instructions (Chen, 2020, 2026). Any difference between the intended fusion and the emitted kernel is a bug in the author's assumptions, not a compiler quirk, because the compiler is allowed to do anything that preserves semantics (OpenAI / triton-lang contributors, 2024; Davies *et al.*, 2025). The dumps should be archived with the benchmark so a later Triton upgrade can be diffed before it is shipped (Chen, 2026; Vellaisamy *et al.*, 2026).

1.3.4 Counting kernels in the serving loop

The codegen review should answer a serving question that no single kernel dump answers: how many Triton kernels, plus eager or vendor kernels, execute for one token step (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). That count is the launches/token counter at the Triton layer, and it is the number capture amortizes (NVIDIA, 2024; Chen, 2026). A fused attention kernel that reduces attention to one launch still leaves projections, MLP, and sampling as separate launches unless the layer integration fuses or captures them (Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). The serving integration should therefore log kernel counts per token per bucket, so a new Triton kernel that adds an unexpected second launch shows up in telemetry before customers feel it (Davies *et al.*, 2025; Chen, 2026). Codegen quality at decode is measured by this count together with bytes/token, never by the PTX beauty of one kernel (Chen, 2020, 2026).

1.3.5 Composing with vendor kernels

Triton kernels rarely run alone in a serving stack; they compose with cuBLAS, cuDNN, or framework kernels around them (Davies *et al.*, 2025; OpenAI / triton-lang contributors, 2024). Each composition boundary inherits the seam rules of this book: if a Triton attention kernel writes its output to HBM so a cuBLAS projection can read it, the fused benefit stops at that edge (Chen, 2026; Dao *et al.*, 2022). The integration review should therefore count the kernels on both sides of the Triton region and verify that capture wraps the whole sequence, not just the Triton portion (NVIDIA, 2024; Vellaisamy *et al.*, 2026). Vendor kernels are opaque to Triton's compiler, so layouts must be agreed by contract at the boundary rather than discovered by inspection (Chen, 2020; Davies *et al.*, 2025). Teams that write beautiful Triton attention and bolt it onto an unfused layer repeat at a smaller scale the graph-level mistake this Part keeps correcting: the kernel is not

the token (Chen, 2026; Davies *et al.*, 2025). A mismatch discovered in telemetry is a boundary-contract bug, not a scheduling accident, and it gets fixed in the integration rather than inside the kernel (Chen, 2026; Davies *et al.*, 2025).

1.4 Triton tuning

Triton's auto-tuning is a decorator, not a search service: `@triton.autotune` takes a list of `triton.Config` entries—block sizes plus `num_warps` and `num_stages`—benchmarks them at runtime for a given key, and caches the winner (OpenAI / triton-lang contributors, 2024). Because the candidate set is author-supplied, the quality of **tun**ing is bounded by which configurations you thought to list. This is leaner than TVM's learned-cost-model search but puts the burden on the engineer to enumerate sensible blocks.

Autotuning is also not free at runtime. Each config in the list must be compiled and benchmarked the first time a new key is seen, so a large config set inflates warmup latency and the first request after a shape change pays the search (OpenAI / triton-lang contributors, 2024). In a serving context that means the config list should be curated for the shapes you actually decode at, not a kitchen-sink sweep—and the cache key must include the dimensions that change the optimal schedule, or the autotuner silently reuses a config tuned for the wrong shape.

The **pitfall** is the operating point you **profile**. The reflexive config list is copied from a GEMM example tuned for large, batched, prefill-shaped matmuls: big `BLOCK_M`/`BLOCK_N`, high `num_stages` (Kwon *et al.*, 2023). At batch-1 decode the attention and projection GEMMs are matrix-vector products, so a large block tile is mostly idle lanes and a deep pipeline over-allocates shared memory for loads that never amortize (Vellaisamy *et al.*, 2026; Chen, 2026). The fix is mechanical: autotune at the decode shape (BS=1, realistic context), include small-block and low-`num_stages` configs, and rank candidates by measured end-to-end ms/token after decomposing bytes/token and launches/token—not by the kernel's isolated GB/s (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). The right config is also **hardware**-specific: a Hopper SKU tolerates a deeper pipeline than a memory-smaller inference GPU, so the cached winner does not port across generations (NVIDIA, 2022).

1.4.1 Designing the config list

Because Triton only benchmarks the configurations the author lists, config-list design is the real tuning skill (OpenAI / triton-lang contributors, 2024; Davies

et al., 2025). A good list spans the plausible trade: small blocks that maximize occupancy, medium blocks that balance occupancy and reuse, and larger blocks that reduce loop iterations, each crossed with two or three pipeline depths and warp counts (Chen, 2026; NVIDIA, 2022). Configs that are obviously wrong for the shape should be omitted rather than benchmarked, because each entry costs compile and run time on every new cache key (OpenAI / triton-lang contributors, 2024; Vellaisamy *et al.*, 2026). The list should also be shape-aware: the attention kernel needs configs tuned to the KV-block dimension, while projection kernels need configs tuned to the hidden dimension (Kwon *et al.*, 2023; Chen, 2026). Serving teams should version the config list with the kernel source so a regression can be traced to a candidate-set change as easily as to a code change (Chen, 2020; Davies *et al.*, 2025).

1.4.2 Warmup, cache keys, and first-token cost

The autotuner's runtime benchmark happens once per cache key, and the first token that observes a new key pays it (OpenAI / triton-lang contributors, 2024; Davies *et al.*, 2025). The cache key must include every dimension that changes the optimal schedule—block structure, context length, head dimension, batch—or the tuner will reuse a winner measured for a different shape (Vellaisamy *et al.*, 2026; Chen, 2026). Serving stacks should warm the cache at startup for every shipped bucket and model configuration, and should treat an unexpected cache miss in production as a first-token SLO event worth alarming on (Kwon *et al.*, 2023; Davies *et al.*, 2025). Offline pre-tuning is preferable: benchmark the config list against recorded decode shapes in CI, write the winning configs into a deployment manifest, and launch serving with autotuning disabled or cache prefilled (Chen, 2020; Vellaisamy *et al.*, 2026). This converts the tuner from a runtime risk into a build-time artifact (Chen, 2026; OpenAI / triton-lang contributors, 2024).

1.4.3 Ranking candidates by the decode contract

The tuner returns whichever candidate its benchmark ranks first, so the benchmark must implement the decode contract rather than kernel GB/s (Davies *et al.*, 2025; Chen, 2026). Rank candidates by end-to-end ms/token at the served context bucket after decomposing bytes/token and launches/token (Vellaisamy *et al.*, 2026; Dao *et al.*, 2022). A candidate that wins the isolated kernel benchmark by ten percent can still lose the layer benchmark if its tile shape forces a layout conversion at the fusion seam or its occupancy starves the following kernel (OpenAI / triton-lang contributors, 2024; Chen, 2026). The benchmark harness should therefore call the

kernel exactly as serving does: same stream, same input pointers, same capture context, same surrounding kernels (NVIDIA, 2024; Davies *et al.*, 2025). Config ranking that stops at the kernel boundary repeats the exact mistake this chapter warns against (Vellaisamy *et al.*, 2026; Chen, 2026).

1.4.4 Re-tuning cadence and cache hygiene

Triton winners decay on the same triggers as other compiler artifacts: Triton and LLVM upgrades, driver changes, model updates, and context-bucket changes (OpenAI / triton-lang contributors, 2024; Chen, 2026; Davies *et al.*, 2025). A nightly job should re-run the decode-cell benchmark against the pinned kernel set and alert when bytes/token or launches/token drift (Vellaisamy *et al.*, 2026; Chen, 2020). When a drift appears, re-autotune the affected kernels over the existing config list and diff the TTGIR before promoting a new winner (OpenAI / triton-lang contributors, 2024; Chen, 2026). Cache hygiene matters equally: stale cache entries for old models or buckets should be pruned, and the shipped cache should contain exactly the kernels the release will execute (Davies *et al.*, 2025; Kwon *et al.*, 2023). Teams without this cadence discover Triton drift through fleet p99, which is the most expensive possible detection point (Vellaisamy *et al.*, 2026; Chen, 2026).

1.5 Triton case study

Make it concrete with a fused decode attention kernel as the **case**. Writing FlashAttention in Triton keeps the query–key–value tiles and the running softmax state on-chip, so the kernel computes attention without materializing the full score matrix to HBM (Dao *et al.*, 2022; Tillet *et al.*, 2019). **Benchmark** it at the decode operating point and the reading matches the book's thesis: an isolated per-operator kernel can win a microbenchmark yet lose end-to-end because each unfused boundary is an HBM round trip and each kernel is a separate launch (Vellaisamy *et al.*, 2026; Chen, 2026).

The arithmetic shows why fusion is the lever. A non-fused attention step writes the full score vector for the new query against the cached context to HBM and reads it back for the softmax; at context length L=2048 in FP32 that score vector is $2048 \times 4 = 8\,\text{KB}$ written and re-read per head per token (Dao *et al.*, 2022). A Triton FlashAttention kernel keeps the running max and sum in registers and never materializes that vector, so across many heads and 32 layers it removes on the order of hundreds of kilobytes of avoidable HBM traffic per token—bytes/token

drops even though the multiply-accumulate count is unchanged (Vellaisamy *et al.*, 2026; Chen, 2026). An isolated GEMM benchmark cannot see this win because it never had the inter-operator round trip in the first place.

This case also points at where Triton meets the book's megakernel thesis. Fusing one attention block is a start, but the larger decode win comes from collapsing a whole decoder layer, or even several, into one persistent kernel that keeps weights and running state resident and issues a single launch per token rather than dozens. Triton makes that practical to express, because the programmer controls exactly where the fusion boundary sits and can widen it until register pressure or shared memory pushes back. The discipline is to grow the fused region while watching occupancy and the launch count together, stopping at the point where one more fused operator would force a spill that costs more than the launch it saved. That trade is measured per token and per hardware generation, never assumed, which is precisely the residency contract the rest of this book builds on.

The **cross-hardware** conclusion is the honest limit of Triton. On the GPU, the fused-and-captured Triton kernel is close to the best available decode path; on a CPU the same tile program is a poor fit for cache-blocked SIMD; on an XDNA/AIE NPU the block-over-warps model does not lower at all and a static-dataflow compiler is required instead (AMD, 2024b,a; Goodfellow *et al.*, 2016). **YiRage** reflects this directly: it uses Triton as its GPU kernel back-end while carrying the Chapter 2 residency rules as checkable IR metadata, so a Triton schedule that would spill a tile between blocks is rejected at compile time, and non-GPU targets are routed to backends whose machine model actually fits (Chen and YiRage Contributors, 2026).

1.5.1 Counter worksheet for fused attention

The fused-attention case produces a worksheet with four rows before it is shippable: the eager baseline's launches/token and bytes/token at the served context bucket, the Triton attention kernel's numbers in isolation, the numbers with the kernel composed into the layer, and the numbers with the layer wrapped in a captured launch sequence (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026). Each row must come from the same model, context, and driver, or the comparison attributes the wrong lever (Chen, 2020; Davies *et al.*, 2025). The worksheet should show where the win comes from: if the isolated kernel is faster but the composed layer is not, the problem is a seam outside the kernel; if capture moves latency but not bytes, orchestration is the remaining lever (Chen, 2026; NVIDIA, 2024). Publishing this worksheet with every Triton kernel PR is how the team keeps "faster kernel" honest about "faster token" (Vellaisamy *et al.*, 2026; Chen, 2026).

1.5.2 Fusion depth and the persistent-kernel decision

The case study stops at a decision boundary: fuse one attention block, fuse a full layer, or go persistent across tokens (Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). Attention-only fusion wins on the KV-loop bytes but leaves projections and MLP as separate launches; full-layer fusion wins on round trips but raises register pressure; a persistent kernel removes even the per-token launch but concentrates failure modes and complicates debugging (Chen, 2026; NVIDIA, 2024). Triton expresses each level with the same block abstraction, so the decision is a measured escalation rather than a rewrite (Tillet *et al.*, 2019; OpenAI / triton-lang contributors, 2024). The escalation should stop at the first level where both bytes/token and launches/token stop improving, because pushing one more operator past that point trades a small launch saving for a spill or occupancy loss (Chen, 2026; NVIDIA, 2022). The decision belongs in the release notes with the worksheet, so the next engineer can see why the stack is not one giant kernel (Chen, 2020; Davies *et al.*, 2025).

1.5.3 Failure signatures for Triton decode

Three recurring failures should be recognized from their profiles. A bytes/token regression with correct-looking TTGIR usually means a layout conversion or register spill that the IR dump exposes only on close reading (OpenAI / triton-lang contributors, 2024; Chen, 2026). A p99 regression that appears only after serving traffic starts usually means cache misses or autotune re-runs on new shape keys, visible as compile stalls in the runtime timeline (Davies *et al.*, 2025; Kwon *et al.*, 2023). A kernel that is fast on an isolated benchmark but slow in the layer usually means the fusion boundary or occupancy interaction sits outside the kernel (Vellaisamy *et al.*, 2026; Chen, 2026). Each signature has a fix path—IR diff, cache warm-up, or layer-level fusion—and none of them is solved by rewriting the kernel arithmetic (Chen, 2020; Davies *et al.*, 2025). Archiving the dumps and worksheet with each release makes the next autopsy a diff instead of a re-derivation (Chen, 2026; Vellaisamy *et al.*, 2026).

1.5.4 Reusing kernels across backends and models

The same Triton attention kernel often serves several models because the block structure depends on tile sizes and head dimensions, not on weights (Dao *et al.*, 2022; OpenAI / triton-lang contributors, 2024). Reuse is safe when the shape signature, dtype, and page layout match; it is unsafe when a model change alters the fusion boundary or the runtime context (Davies *et al.*, 2025; Chen, 2026). Kernel

libraries should therefore expose the shape contract explicitly and benchmark each (model, bucket, SKU) combination against the shared kernel rather than assuming one autotune winner generalizes (Vellaisamy *et al.*, 2026; Chen, 2020). A reused kernel that was tuned for one head dimension can still run correctly but suboptimally on another, and correctness hides that inefficiency (OpenAI / triton-lang contributors, 2024; Davies *et al.*, 2025). The case study's final artifact is a small kernel library with per-signature winners, an archived worksheet per signature, and a serving layer that selects the winner from telemetry keys (Chen, 2026, 2020).

1.5.5 Worked counter expectations

A realistic Triton attention kernel should change counters in predictable directions. Compared with an unfused eager baseline, the kernel should reduce bytes/token because score and probability materialization disappear; it should reduce kernels per token because the KV loop, softmax, and accumulation collapse into one launch; and it should keep ms/token within the range the memory-bound floor predicts once launches are captured (Dao *et al.*, 2022; Chen, 2026; Vellaisamy *et al.*, 2026). If bytes/token drops but ms/token does not, host dispatch or a neighboring seam is the remaining cost; if ms/token drops but bytes/token is flat, the win is orchestration rather than residency (Davies *et al.*, 2025; NVIDIA, 2024). These expectations make benchmark review mechanical: a PR that claims a Triton attention win must show which counter moved, and the answer determines whether the PR belongs in the residency chapter of this book or the orchestration chapter (Chen, 2026, 2020). Publishing the expectations next to the worksheet keeps reviewers and authors speaking the same counter language (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

1.6 Key Takeaways

- **Triton is a GPU block compiler, not a universal target.** Its tile/SIMT model maps to NVIDIA PTX and AMD AMDGCN but not to static-dataflow NPUs, so treat it as one back-end in a multi-hardware stack (Tillet *et al.*, 2019; OpenAI / triton-lang contributors, 2024). The build matrix is GPU-shaped (AMD, 2024b).
- **Win residency in the TTGIR passes.** Layout assignment, coalescing, and software pipelining at the TritonGPU dialect decide whether tiles stay on-chip; a bad layout pass spills silently and shows up only as a bytes/token

regression (OpenAI / triton-lang contributors, 2024; Chen, 2026). Dump and diff the IR before merge (Vellaisamy *et al.*, 2026).

- **Grade codegen by orchestration, not one kernel.** At batch-1 the launch floor and HBM round trips dominate, so fuse the decoder layer and capture the launch sequence; capture alone recovered roughly 1.26× on an H100 decode cell (Chen, 2026; NVIDIA, 2024). A fast standalone MMA is not a fast token (Dao *et al.*, 2022).
- **Autotune at the decode operating point.** `@triton.autotune` only explores the configs you list; default GEMM configs over-tile and over-pipeline for batch-1 (OpenAI / triton-lang contributors, 2024; Kwon *et al.*, 2023). Include small-block, low-`num_stages` configs and rank by ms/token (Davies *et al.*, 2025).
- **Tuned kernels do not port across GPUs.** Optimal `num_warps`/`num_stages` depend on shared-memory size and async-copy depth, so a cached winner must be re-tuned per generation (NVIDIA, 2022; SemiAnalysis, 2024). Portability is a re-tune, not a copy (Goodfellow *et al.*, 2016).

1.7 Conclusion

Triton's tile abstraction and MLIR dialect pipeline are the cleanest way to write fused, residency-aware GPU kernels at Python productivity, and they are why decode-critical kernels like FlashAttention are now maintainable (Tillet *et al.*, 2019; Dao *et al.*, 2022). But the payoff is conditional: Triton helps interactive serving only when its TTGIR passes, its PTX/AMDGCN codegen, and its autotuner are aimed at the batch-1 decode shape and graded on bytes/token and launches/token (Vellaisamy *et al.*, 2026; Chen, 2026). That is the residency contract YiRage enforces as IR metadata across its backends (Chen and YiRage Contributors, 2026). Triton also exposes the limit of a GPU-only kernel compiler, which motivates the next surface: Chapter **??** turns to IREE, an MLIR-native compiler and runtime that carries the same dialect-lowering discipline across a wider hardware matrix (Goodfellow *et al.*, 2016).

Bibliography

AMD (2024a). Ai engine technology. `https://www.AMD.com/en/products/adaptive-socs-and-fpgas/technologies/AI-engine.html`. 2, 14

AMD (2024b). AMD XDNA architecture. `https://www.AMD.com/en/technologies/XDNA.html`. 2, 4, 9, 14, 16

Chen, J. (2026). Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode. *arXiv preprint arXiv:2605.30571*. 44-cell cross-GPU study; CUDA Graphs A/B on H100 and L4. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17

Chen, X. (2020). Compiler-based hardware profiling for software-defined architecture. 3, 5, 6, 7, 9, 10, 12, 13, 14, 15, 16

Chen, X. and YiRage Contributors (2026). Yirage: Multi-backend LLM inference optimization. `https://github.com/chenxingqiang/YiRage`. Yield Revolutionary AGile Engine; extends Mirage with CUDA, CPU, MPS, Triton, NKI, and MLIR backends. 2, 9, 14, 17

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*. 1, 3, 8, 10, 12, 13, 15, 16, 17

Davies, M. *et al.* (2025). Liminal: Exploring the frontiers of LLM decode performance. *arXiv preprint arXiv:2507.14397*. Limit study; DeepSeek-V3 provisions 10× more decode than prefill nodes. 1, 3, 4, 5, 6, 7, 9, 10, 11, 12, 13, 14, 15, 16, 17

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. 2, 4, 11, 14, 17

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*. 6, 7, 9, 11, 12, 13, 15, 17

NVIDIA (2022). Nvidia hopper architecture in-depth. NVIDIA Technical Blog. 2, 3, 4, 6, 7, 8, 9, 11, 12, 15, 17

NVIDIA (2024). Cuda graphs. CUDA Programming Guide. 1, 5, 8, 10, 13, 14, 15, 16, 17

OpenAI / triton-lang contributors (2024). Triton: A language and compiler for custom deep-learning primitives. `https://github.com/triton-lang/triton`. MLIR-based backend: Triton IR (TTIR) → TritonGPU IR (TTGIR) → LLVM → PTX/AMDGCN; `@triton.autotune` with num_warps/num_stages. 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 15, 16, 17

OpenXLA Project (2024). XLA: Accelerated linear algebra — OpenXLA architecture. `https://openxla.org/xla/architecture`. StableHLO → HLO → LLVM/PTX/TPU; operator fusion as primary optimization (no HBM intermediates); PriorityFusion / Multi-Output passes; CommandBufferThunk launch capture; Triton emitters. 3

SemiAnalysis (2024). Nvidia tensor core evolution: From volta to blackwell. Newsletter. 2, 4, 5, 6, 7, 8, 17

Tillet, P., Kung, H. T., and Cox, D. (2019). Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL@PLDI)*, pages 10–19. Tile = statically shaped sub-array; Triton-C + LLVM-based Triton-IR; matmul/conv on par with cuBLAS/cuDNN. 1, 2, 3, 4, 5, 6, 13, 15, 16, 17

Vellaisamy, P., Tripathi, S., Natarajan, V., Thenarasu, S. S., Blanton, S., and Shen, J. P. (2026). Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition. *arXiv preprint arXiv:2603.12465*. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17

Index

Tile programming model, 1
Triton, 1
TritonGPU IR, 1